

Лабораторная работа 13

Проектирование и разработка библиотек под Composer.

Основы разработки универсальных пакетных решений.

Контрольные вопросы

1. Для чего используется поставщик услуг при разработке пакета?

- Для регистрации зависимостей, команд, маршрутов, миграций и др. рес-ов пакета в приложении Laravel.

2. Как автоматически зарегистрировать пакет при его установке?

- Через секцию extra в composer.json с указанием класса поставщика услуг:

```
"laravel": { "providers": [ "Vendor\\Package\\PackageServiceProvider" ] }
```

3. Что такое фасады в Laravel и какова цель их использования?

- Фасады - это статические прокси-классы, обеспечивающие удобный доступ к сервисам контейнера. Цель - упростить вызов методов сервиса.

нее статических статических методов.

4. Как регистрируются созданные команды в пакете?

- В методе `boot()` поставлена услуга `возврат $this->commands([Command::class])`.

5. Как правильно конфигурировать параметры пакета?

- Создать конфигурационный файл, загрузить его в методе `boot()` через `$this->mergeConfigFrom()`, а затем позволить опубликовать через `php artisan vendor:publish`.

6. Как создать экспортируемый файл конфигурации? - Поместить файл в `config/пакет.php`, а в поставке услуг в методе `boot()` `возврат $this->publishes([DIR: './config/пакет.php' => config_path('пакет.php')])`.

7. Описать способ публикации миграций в пакете: 1) Создать файл конфигурации внутри вашего пакета (например, `config/миссифиг.php`), кот. возвращает массив с параметрами.

2) В классе поставщика услуг (Service Provider) вашего пакета внутри метода `boot()` вызвать метод `publishes()`.

3) В `publishes()` указать путь к вашему конфигурационному файлу (`DIR__ .'/config/myconfig.php'`) → конкретный путь в проекте (`config-path('myconfig.php')`).

4) После установки пакета пользователю выполнить команду `php artisan vendor:publish --tag^...`, и файл скопируется в папку `config` его `Laravel`-приложения.

8. В чем разница между публикацией и миграцией и их загрузкой непосредственно из пакета?

- При загрузке миграции выполняются автоматически при установке/обновлении, но не попадают в `database/migrations` проекта. При публикации файлы копируются в проект, и пользователь сам управляет миграциями.

9. Описать процесс регистрации маршрутов

пакета в проекте.

- 1) Создать файл маршрутов в папке вашего пакета, где опишите нужные маршруты, используя контроллеры пакета;
- 2) В методе `boot()` вашего поставщика услуг (`ServiceProvider`) вызовите метод `loadRoutesFrom()`.
- 3) Передайте в метод полный путь к файлу маршрутов: `_DIR_ . '/routes/web.php'`.
- 4) После установки пакета в проект Laravel маршруты автоматически подключатся — не нужно выполнять `php artisan vendor:publish` для маршрутов, они загружаются напрямую из пакетов.

10. Описать процесс публикации пакета:

- Создать репозиторий на GitHub.
- Настроить `composer.json` (имя, зависимости).
- Реализовать поставщика услуг и зарегистрировать рес-сы.
- Создать релиз на GitHub.
- Зарегистрироваться на `packagist.org`.
- Отправить URL GitHub-репозитория на `packagist`.